



University
of Regina

Go far, *Together.*

ENSE 375 – Software Testing and Validation

Group Leader

James Clinton 200268717

Peyton Bowen 200504005

Jaydon Forrester

Table of Contents

1	Introduction.....	5
2	Design Problem.....	6
2.1	Problem Definition.....	6
2.2	Design Requirements.....	6
2.2.1	Functions.....	6
2.2.2	Objectives.....	6
2.2.3	Constraints.....	6
3	Solution.....	7
3.1	Solution 1.....	7
3.2	Solution 2.....	7
3.3	Final Solution.....	7
3.3.1	Components.....	7
3.3.2	Environmental, Societal, Safety, and Economic Considerations.....	7
3.3.3	Test Cases and Results.....	7
3.3.4	Limitations.....	7
4	Team Work.....	8
4.1	Meeting 1.....	8
4.2	Meeting 2.....	8
4.3	Meeting 3.....	8
4.4	Meeting 4.....	8
5	Project Management.....	9
6	Conclusion and Future Work.....	10
7	References.....	11
8	Appendix.....	12

- Proof read the text for typing and grammar mistakes.

- Follow the IEEE Bibliography style for the references by selecting "References/ Citations & Bibliography/ Style".

List of Figures

Figure 1. Project Management Gantt Chart.....	19
---	----

List of Tables

Table 1. Meeting 1.....	16
Table 1. Meeting 2.....	17
Table 1. Meeting 3.....	17

1 Introduction

- Give a brief description of the design and a summary of the relevant background information related to the topic. Give a rationale about what is needed and why.
- Give the reader an overview of what is in the next sections.
- Do not put any detailed results of your work here.

2 Design Problem

This section has the following two subsections:

2.1 Problem Definition

For this design project, our group defined the problem to be the process of planning and designing, then creating and thoroughly testing a Model View Controller website using proper industry standard testing practices covered in class. Constraints also apply, which we must consider and follow several of them; these include: Economic factors, Regulatory compliance, Reliability, Sustainability and Environmental Factors, Ethics, and Societal Impacts. Due to the nature of the project, the focus of the project is to gain practical experience using several different testing and validation techniques, including Boundary value testing, Equivalence class testing, Decision tables testing, State transition testing, and Use case testing. As well, two functions must be tested; using Path testing and Data flow. During this process, we will follow the structured engineering design process; such as considering requirements, research, and brainstorming. Including analyzing and iterating on potential functions and prototypes. One important idea we are considering is whether to have accounts, which has a large upside, and some downside to implementing, and our process will prototype and test it. Our approved topic/idea for this project is Group Leader, a scheduling app for groups. Each member adds their schedule and availability, then the program will find common times between them, allowing the group to all pick a preferred time. This has a lot of use cases, including just for friend groups to meet, or even in workplaces, when they would need to schedule a meeting. During the development cycle, we will document this process thoroughly, including test cases we use, commenting code, and having a public Github. Delegating work is another

area for this project, where we will track which group member contributes individually.

2.2 Design Requirements

This section has the following three subsections:

2.2.1 Functions

- The system will perform the following functions:
- **Create and manage user accounts** to allow users to sign in and maintain personal schedules.
- **Collect and update weekly availability** through a reusable master schedule or a group-specific schedule.
- **Record meeting preferences** and distinguish between available, available if necessary, and unavailable time slots.
- **Create and manage events** with a group name, meeting purpose, duration, candidate dates, and permitted meeting hours.
- **Generate shareable invitation links** to allow participants to access events and submit availability.
- **Track availability submissions** and identify missing responses as unknown.
- **Calculate and rank suitable meeting times** based on participant availability and preferences within the event's scheduling constraints.

- **Display group availability** through a heatmap that shows participant availability for each time slot.
- **Confirm and display meeting details** after the organizer selects a final meeting time.
- **Repeat the scheduling process automatically** at specified intervals for recurring events.

2.2.2 Objectives

- The objectives of the Group Leader application are as follows: we aim to be useful, intuitive, and quick. Firstly, by our objective being useful, we mean that our website should be providing meaning or practical information that was not available before using our service. This means that, for a team of 20, when each member receives a link they can add their schedule, see how many people out of the total have added theirs, and the current best times for the group to meet. With this information, it is useful to them as they do not have to ask everyone their schedule, they do not have to figure out manually what time fits for everyone, and it can be done asynchronously. Thus, this information is of some value and saves time. The second objective we aim to meet is to be intuitive. What being intuitive to us means, is that the website should be easy to use and add their schedule. The website shows lots of useful information such as: how many people have added their schedule thus far, group name and meeting purpose, and the current best time. I also believe that when the group leader creates the schedule link, he can input the possible time the algorithm will select, so for an office group, it will not schedule the meeting at 2am nor display that time. This adds intuitiveness as the algorithm can do more work for the group, and be able

to deliver better information to the group. The third objective we will consider is to be quick, where we aim to allow information to be inputted as fast as possible, we can aim to do this through using buttons over text inputs, design choices, and iterating on design to see what UX tester responds to. As well, considering the entire process is key to being quick, such as from when the first person creates the group, to when everyone receives it. For a lot of people, this could be a longer process, so it is important for it to be easy and fast for each person to enter their schedule. Overall, these objectives combine to produce an easy, intuitive, quick, and useful application that provides value to the user through new information they can use in their daily lives.

2.2.3 Constraints

- **Economic Factors** - Our project is free/inexpensive to complete, but must be fully completed by December 3, with a check-in on October 8. This limits our scope by limiting how much time we have to build our project.
- **Regulatory Compliance (Security & Access)** - If Group Leader is to have any identifiable or personal information (such as with accounts), it is a high priority to minimize risks of vulnerabilities in the software.
- **Reliability** - We need to ensure that Group Leader is reliable, and that there will not be problems. It should be able to successfully complete all needed functions without issue.
- **Sustainability & Environmental Impacts** - Group Leader should be easily sustained without any noticeable footprint.

- **Ethics** - The project must not cross any ethical boundaries. A focus on safety/security should solve any ethical problems with the design, but that does not mean other issues cannot arise and we will plan our project to ensure this is not a problem.
- **Societal Impacts** - The project itself is relatively light, when it comes to societal impacts either way. One potential issue is misuse of data the site is given, which we will ensure is not going to happen.

3 Solution

In this section, you will provide an account of some solutions your team brainstormed to implement and test the project. Some solutions might not have all the desired features, some might not satisfy the constraints, or both. These solutions come up in your mind while you brainstorm ways of implementing all the features while meeting the constraints. Towards, the end you select a solution that you think has all the features, testable and satisfies all the constraints. Remember that an engineering design is iterative in nature!

3.1 Solution 1

A website that allows for a user to create and send a link to fill out a weekly schedule, which anyone with the link can do without so much as logging in. This schedule allows for various types of options, such as green (good), yellow (complicated), red (not going to work out), and white (indifferent).

This design is meant to allow for weekly planning, determining what times everyone is free every week in order to more easily make decisions on when works best for everyone. The varied colors are intended to allow for more specific clarification, for example, if you are a student, and you know that next semester you have a class on a time you are currently free, you'd mark it yellow and clarify.

This design got scrapped early on as it is simply too complex and would be quite difficult to visualize at a glance when combining the schedules into one visual. In this solution, testing is also rather complicated, with so many possible options, which may be a bit out of our current scope.

3.2 Solution 2

Our solution is a schedule website allowing for groups to schedule events and view the best time for people to meet, when they have complicated schedules. The website needs to be quick, intuitive, and easy to use. When the user opens the web-page, there should be minimal clutter, simply presented with two choices: create a meeting, or join one. Joining requires a code from the admin of the meeting however. When someone creates a meeting, they select the type of hours the meeting can occur: i.e. evenings, night, business hours, or custom. As well, the number of people in the meeting, and they are provided a link and code to send to others in the meet. When joining the meeting, the joiner will provide his availability by selecting the times they are available. This design slightly differs from solution 1, and is simplified to only be a binary available or not available. This sends the data to the database, which updates the hosts master link, where the host can then select a time the group meets. The joiners can then reopen the link to see the selected official meeting time. There will also be an option to select if this meeting is weekly, or one time. Once everyone has added their schedule, we may create an algorithm to suggest the best time. This meets most if not all of our functions, and fits within our constraints of being ethical and having only positive societal impacts.

The basic architecture we plan on creating for the site is to host a MVC application which will pull from a mysql database. On the database, each meet will use a hex code as a primary key, which will also be the path for the sharable link of the meet. We will store times people are unavailable in the database, then when combining people's schedules, we will gray out any block of time where people are unavailable. Also stored is if this unavailability will be repeated weekly. Since we

could host the website and database locally on our machine, this architecture solution meets this constraint. In this solution, accounts are not present, so we do not have to consider regulatory compliance. For reliability, we could see to it with testing, that the program is reliable.

3.3 Final Solution

Our final solution builds upon Design 2 by introducing user accounts, improved scheduling functionality, and an algorithm to determine the best available meeting times. The website will allow users to create groups, manage their weekly availability, and schedule meetings based on the availability of group members. The goal is to create a system that is intuitive, reliable, and easy to use while addressing the limitations identified in our previous designs.

One of the main deficiencies of Design 2 was that users would have to enter their availability every time they joined a new meeting. To address this, Design 3 introduces user accounts that allow individuals to save and manage their weekly availability. Users will be able to maintain a master schedule while also adjusting their availability for specific groups or meetings. This reduces repetitive data entry and makes the system more practical for groups that meet regularly.

Another deficiency of Design 2 was that it relied on the meeting host to manually review everyone's availability and select a meeting time. In Design 3, we will implement a scheduling algorithm that evaluates the availability of all participants and ranks possible meeting times. The algorithm will prioritize times when the greatest number of participants are available, while also considering preferred

meeting times when provided. This will allow the host to make an informed decision without having to manually compare individual schedules.

Design 3 will also improve how recurring meetings are handled. Rather than requiring users to repeatedly create new meetings, the system will support recurring events that automatically reevaluate participant availability for each occurrence. Users will also be able to create one-time events, invite other members, and view confirmed meeting times within their groups

From a testing perspective, Design 3 provides a more structured and testable solution. The scheduling algorithm can be tested independently using predefined availability datasets to verify that it correctly identifies and ranks meeting times. Unit tests can be used to verify individual functions, such as availability updates, conflict detection, and recurring event calculations. Integration testing can verify that user accounts, groups, events, and the database interact correctly. Finally, usability testing can help determine whether users can easily create meetings, enter their availability, and understand the suggested meeting times.

3.3.1 Components

What components you used in the solution? What is the main purpose of using individual component? What testing method did you employ for each component? Provide a block diagram (with a numbered caption, such as Fig. 1) representing the connectivity and interaction between all the components.

3.3.2 Environmental, Societal, Safety, and Economic Considerations

Explain how your engineering design took into account environmental, societal, economic and other constraints into consideration. It may include how your design has positive contributions to the environment and society? What type of economic

decisions you made? How did you make sure that the design is reliable and safe to use?

3.3.3 Test Cases and results

What test suits did you design to test your prototype? How did you execute the test cases to test the prototype?

3.3.4 Limitations

Every product has some limitations, and so is the case with your design product. Highlight some of the limitations of your solution here.

4 Team Work

Since this is a group project, you must have a fair distribution of tasks among yourselves. To this end, you must hold meetings to discuss the distribution of tasks and to keep a track of the project progress.

4.1 Meeting 1

Time: September 17 2026, around 1 pm.

Agenda: Distribution of Project Tasks

Team Member	Previous Task	Completion State	Next Task
James Clinton	N/A	Project Definition, Github creation, Project File Template	2.2.2 Objectives
Peyton Bowen	N/A	Business Case	2.2.1 Functions
Jaydon Forrester	N/A	Cost/Benefit	2.2.3 Constraints

4.2 Meeting 2

Time: Virtual Meeting – Throughout 23 September to 24 September

Agenda: Review of Individual Progress

Team Member	Previous Task	Completion State	Next Task
James Clinton	Project Definition, Github creation, Project File Template	2.2.2 Objectives100%	3.2, Project File Template
Peyton Bowen	Business Case	2.2.1 Functions – 100%	3.1
Jaydon Forrester	Cost/Benefit	2.2.3 Constraints 100%	3.3, Github

4.3 Meeting 3

Time: Throughout Oct 8 2026, Oct 10 2026

Agenda: Completing Designs

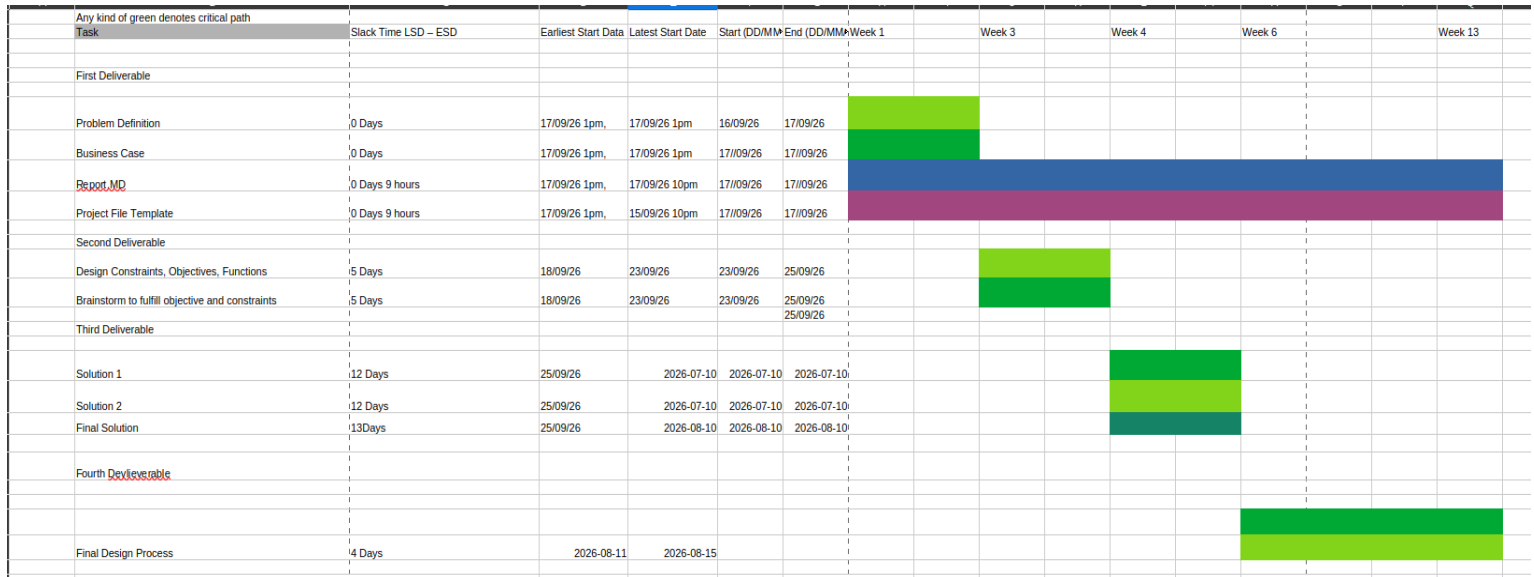
Team Member	Previous Task	Completion State	Next Task
James Clinton	2.2.2 Objectives100%	3.2, Project File Template 100%	Section 3.3
Peyton Bowen	2.2.1 Functions – 100%	3.1 100%	Section 3.3
Jaydon Forrester	2.2.3 Constraints 100%	3.3, Github 100%	Section 3.3

4.4 Meeting 4

Provide a similar description here.

5 Project Management

Provide a Gantt chart showing the progress of your work here. Mention all the tasks along with their predecessors. Provide the slack time of each task and identify the critical path.



6 Conclusion and Future Work

- A summary of what you achieved. Mention all the design functions and objectives that you achieved while satisfying testing requirements?
- While keeping the limitations of your solution, provide recommendations for future design improvements.

7 References

- Use the IEEE reference style.
- Do not put any reference if it is not cited in the text.

8 Appendix

If you want to provide an additional information, use this appendix.